



UNIVERSITATEA TEHNICĂ

DIN CLUJ-NAPOCA

**FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE**

TITLUL LUCRĂRII DE DISERTAȚIE

LUCRARE DE DISERTAȚIE

Absolvent masterand: **Prenume NUME**

Coordonator științific: **titlul științific Prenume NUME**

Iulie, 2026



UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE

DECAN
Prof.Dr.Ing. Vlad MUREȘAN

DIRECTOR DEPARTAMENT
Prof.Dr.Ing. Rodica POTOLEA

Student masterand: **Prenume NUME**

TITLUL LUCRĂRII DE DISERTAȚIE

1. **Enunțul temei:** *Scurtă descriere a temei lucrării de disertație și datele inițiale*
2. **Conținutul lucrării:** *(enumerarea părților componente) Exemplu: Pagina de prezentare, aprecierile coordonatorului, titlul capitolului 1, titlul capitolului 2, ..., titlul capitolului n, bibliografie, anexe.*
3. **Locul documentării:** *Exemplu: UTCN, Cluj-Napoca*
4. **Consultanți:**
5. **Data emiterii temei:** *Exemplu: noiembrie 2025*
6. **Data predării:** *Exemplu: 5 iulie 2026*

Student masterand:

Coordonator științific:



UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE**Declarație pe proprie răspundere privind
autenticitatea lucrării de disertație**

Subsemnatul(a) Prenume NUME, legitimat(ă) cuserianr., CNP, autorul lucrării TITLUL LUCRĂRII DE DISERTAȚIE, elaborată în vederea susținerii examenului de finalizare a studiilor de disertație la Facultatea de Automatică și Calculatoare, Programul de studii, din cadrul Universității Tehnice din Cluj-Napoca, sesiunea (iulie/septembrie) a anului universitar 2025-2026, declar pe proprie răspundere că această lucrare este rezultatul propriei activități intelectuale, pe baza cercetărilor mele și pe baza informațiilor obținute din surse care au fost citate în textul lucrării și în bibliografie.

Declar că această lucrare nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor.

Declar, de asemenea, că această lucrare nu a mai fost prezentată în fața unei alte comisii de examen de disertație.

În cazul constatării ulterioare a unor declarații false, voi suporta sancțiunile administrative, respectiv, *anularea examenului de disertație*.

Sunt de acord ca, pe tot parcursul vieții, în cazul în care este necesar și se va dori verificarea autenticității lucrării mele să fiu identificat și verificat în baza datelor declarate de mine.

Data

.....

Prenume NUME

.....

Semnătura

Cuprins

Capitolul 1	Introducere	1
Capitolul 2	Obiectivele cercetării	6
Capitolul 3	Studiu bibliografic/Stadiul actual în domeniu	9
3.1	Distcc	9
3.2	Ccache	11
3.3	Sccache	13
3.4	DiSCo	13
Capitolul 4	Prezentarea proiectului	15
4.1	Fundamentare teoretică	15
4.1.1	Sisteme distribuite	15
4.1.2	Mecanisme de concurență și paralelism	17
4.1.3	Sisteme de build pentru C/C++	22
Capitolul 5	Rezultate teoretice și experimentale	25
Capitolul 6	Concluzii	26
6.1	Conținut	26
6.2	Detalii tehnice	26
6.2.1	Dimensiune	26
Bibliography		27
Anexa A	Pseudo-cod sau cod (dacă există)	29

Capitolul 1

Introducere

Limbajele de programare compilate, precum **C** și **C++**, sunt de cele mai multe ori alegerea corectă pentru dezvoltarea de sisteme și aplicații pentru care performanța în timpul rulării este un aspect de importanță critică, însă alegerea lor prezintă inconveniente în ceea ce privește procesul de build. Cerințele ridicate de memorie și timpii îndelungați de compilare cresc proporțional cu dimensiunea proiectului aflat în dezvoltare.

Timpii îndelungați de compilare sunt de natură să încetinească drastic procesul de dezvoltare, ceea ce determină întârzieri ce afectează livrarea produsului către clienți și pierderi costisitoare de timp pentru evoluția proiectului.

Evident, investiția în hardware mai performant pentru a reduce timpul de compilare poate diminua magnitudinea problemei, însă se estimează că în următorii zece ani, puterea de procesare se va satura (legea lui Moor nu va mai fi valabilă începând din 2036 [1]), iar accentul se va muta asupra optimizării resurselor de rețea. Progresele în domeniul comunicațiilor vor permite transferuri rapide și eficiente de date între nodurile sistemelor distribuite, susținând astfel implementarea și extinderea proceselor distribuite de compilare. Această abordare permite fragmentarea și procesarea paralelă a sarcinilor de build pe mai multe noduri din rețea, fie ele stații locale, servere dedicate sau resurse din cloud. Drept rezultat, proiectele software vor putea beneficia de timpi reduși de compilare și de o utilizare mai eficientă a resurselor disponibile, contribuind la creșterea productivității și la reducerea blocajelor din ciclurile de dezvoltare.

Compilarea folosind mai multe sisteme de calcul care comunică prin rețea este susținută și de faptul că, odată cu răspândirea utilizării generale a calculatoarelor și a resurselor cloud, timpii de neutilizare ai sistemelor (Idle Times), au crescut considerabil. În instituții educaționale, calculatoarele au o durată de neutilizare medie de 28% (adică 2672 de ore din total de 9550 de ore de funcționare) [2]. Așadar, exploatarea resurselor de calcul ale sistemelor neutilizate reprezintă o eficientizare binevenită pentru reducerea duratei de build.

Pentru a putea înțelege de ce durata procesului de compilare pentru **C/C++** (dar mai ales pentru **C++**) este semnificativ mai mare față de alte limbaje de programare, găsim esențial să explicăm modul de desfășurare al acestui proces.

Transformarea codului sursă într-un fișier executabil este compusă din trei etape majore. Primul pas îl reprezintă expandarea directivelor de preprocesare, moment în care dependențele dintre fișierele sursă ale proiectului sunt determinate. Rezultatul etapei de preprocesare îl reprezintă o colecție de unități de translație (Translation Units), fiecare fișier sursă având asociat un astfel de **translation unit**. Urmează etapa de compilare, în urma căreia unitățile de translație sunt transformate în fișiere ce conțin cod mașină, numite fișiere obiect (Object Files). În ultima etapa, numită link-editare, toate fișierele obiect rezultate în urma compilării sunt conectate pentru a genera fișierul executabil al proiectului [3].

Etapa de preprocesare este [3] responsabilă de pregătirea fișierelor sursă pentru a fi furnizate ca intrare pentru compilator. În **C/C++**, fiecare simbol poate fi definit o singură dată, însă declararea simbolului poate să apară de mai multe ori, întrucât fiecare simbol utilizat într-o unitate de translație, trebuie să aibă o declarație locală. Această chestiune este tratată de mecanismul de includere a fișierelor, folosind directiva de preprocesare **#include**. Figura 1.1 exemplifică mecanismul de preprocesare al fișierelor sursă.

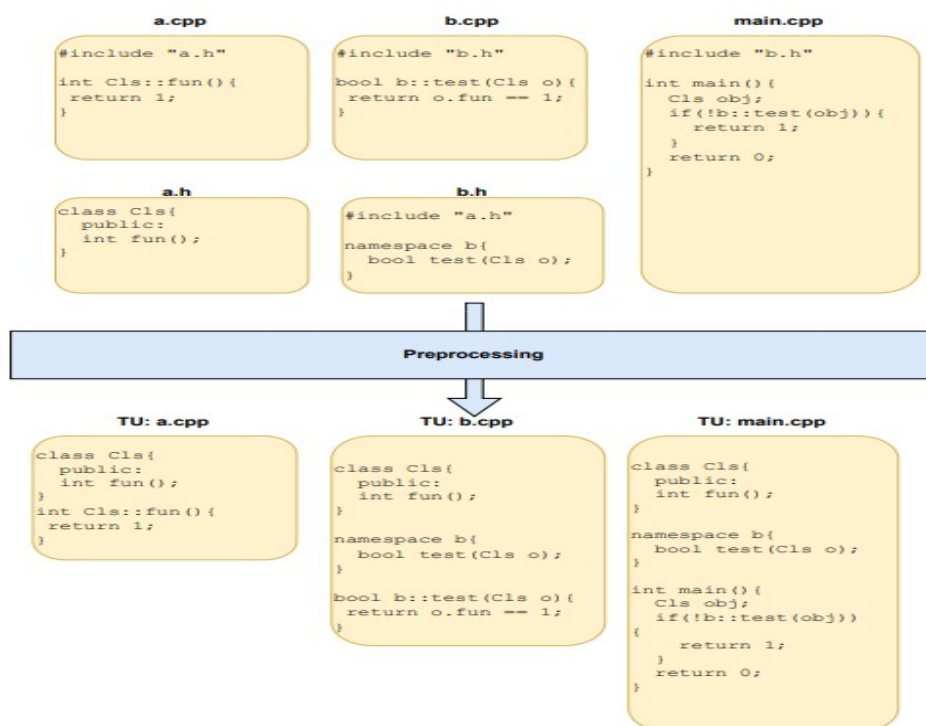


Figura 1.1: Preprocesarea fișierelor sursă [3]

Unitățile de translație sunt furnizate ca date de intrare pentru compilator, care detectează erorile de sintaxă și produce fișierele obiect, care deși conțin cod mașină, nu pot fi încă executate. Figura exemplifică etapa de compilare.

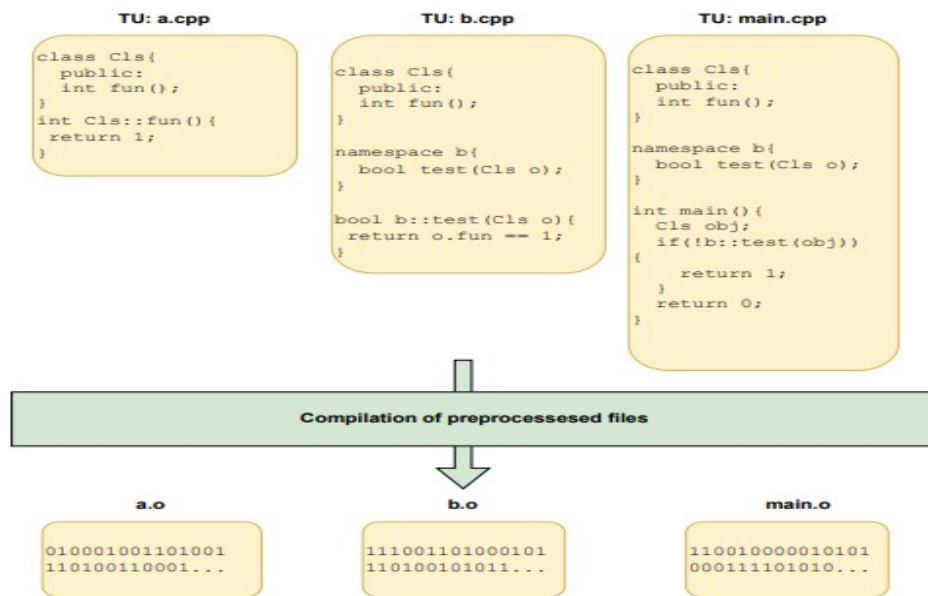


Figura 1.2: Compilarea unităților de translație [3]

Etapa de linking [3] conectează fișierele obiect pentru ca declarațiile locale să fie disponibile în celelalte unități de translație. Ieșirea linker-ului reprezintă un fișier executabil, care este rezultatul final al procesului de build.

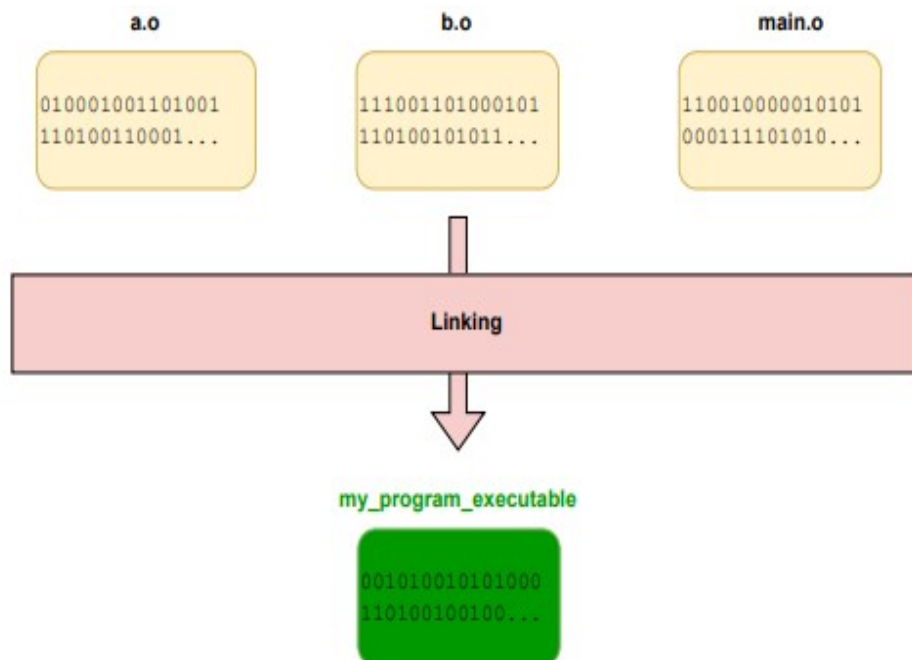


Figura 1.3: Generearea fișierului executabil [3]

Proiectele complexe, în care numărul de dependențe este mare, suferă des de recompilarea unui număr mare de fișiere, ceea ce crește semnificativ durata de compilare. Motivul pentru care compilarea excesivă este greu de evitat îl reprezintă faptul că, dacă un fișier header care este inclus de un număr mare de fișiere sursă este modificat, atunci se declanșează recompilarea tuturor fișierelor sursă care depind de acel header. Când numărul de fișiere header incluse este mare, există șansa să se creeze cicluri de dependență, o situație în care mai multe fișiere se includ reciproc [4].

Totodată, mecanismul de **templating** din **C++**, permite definirea de clase și funcții generice, care pot fi instanțiate folosind tipuri de date diverse. Întrucât **C++** este un limbaj de programare cu sistem static de tipuri (statically typed), de fiecare data când un template este invocat pentru un nou tip de argument, codul pentru implementarea specifică acelui tip de date este generat în mod automat de către compilator [5]. În cazul în care template-ul este instanțiat pentru foarte multe tipuri de date diferite, compiler-ul generează un volum semnificativ de cod repetitiv, ceea ce poate încetini procesul de build [6].

Pentru a atrage atenția asupra scenariilor în care timpul de build devine o problemă, voi invoca creșterea exponențială a volumului de cod sursă care intră în componența unor proiecte open-source cunoscute, precum **Linux Kernel** și **Android**[7]. Dimensiunea codebase-ului pentru acestea se ridică la ordinul zecilor de milioane de linii de cod. Creșterea volumului de cod de-a lungul timpului este vizibilă în Figura 1.4.

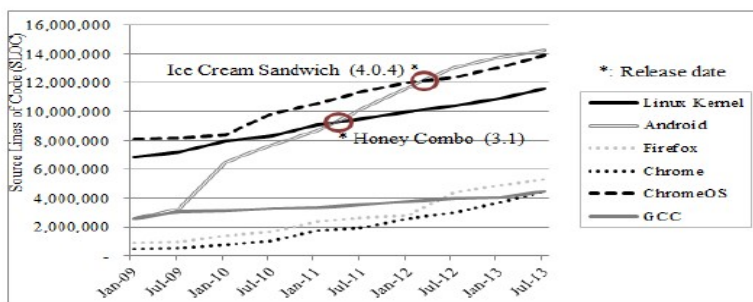


Figura 1.4: Creșterea codebase-ului de-a lungul anilor pentru câteva proiecte open-source [8]

Pentru sistemul de operare mobil **Android 4.2.2**, timpul de build total ajunge la 51 de minute [8].

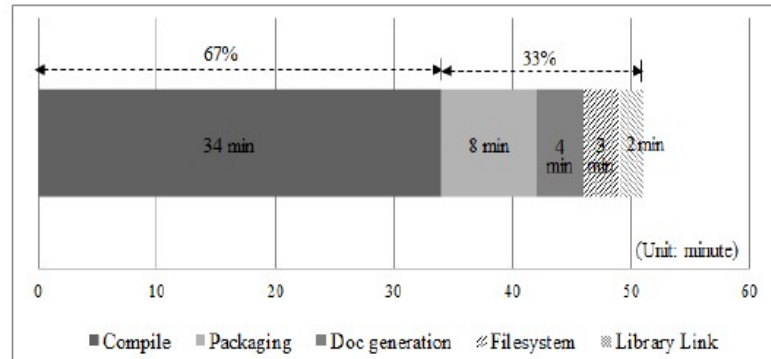


Figura 1.5: Timp de build Android [8]

Capitolul 2

Obiectivele cercetării

Obiectivul principal al cercetării constă în evaluarea tehnologiilor moderne de comunicare din domeniul sistemelor distribuite, în scopul implementării unui nou sistem de compilare distribuită pentru proiecte C/C++, care să ofere performanțe competitive în ceea ce privește eficientizarea procesului de compilare, comparativ cu alternativele deja existente. Sistemul dezvoltat propune o abordare eficientă, scalabilă și flexibilă din punct de vedere arhitectural, de a crește viteza procesului de compilare, fără a introduce incompatibilități cu uneltele deja folosite în dezvoltare proiectelor.

Compilarea proiectelor dezvoltate folosind C/C++ este compusă din etape multiple de transformare a codului sursă: preprocesarea, compilarea, generarea de cod mașină pentru platforma vizată și link-editarea. Unele operațiuni aferente procesului de compilare pot fi executate independent, în paralel, însă gestiunea dependențelor dintre fișierele componente ale proiectului, artefactele generate și setările de configurare ale compilatorului introduce un grad însemnat de complexitate. Un obiectiv ce trebuie îndeplinit de către orice sistem distribuit de acest tip îl reprezintă identificarea corectă a sarcinilor care pot fi preluate de către sistemele de calcul auxiliare, spre a fi paralelizate, astfel încât să fie menținută corectitudinea și capacitatea de reproducere facilă a procesului de compilare.

Lucrarea investighează modul în care sarcinile de compilare pot să fie distribuite într-o manieră eficientă către mai multe noduri de calcul din rețea, pentru a se minimiza suprautilizarea resurselor de calcul, precum și a canalelor de comunicare. De asemenea, sistemul trebuie să păstreze în evidență disponibilitatea sistemelor ce contribuie la procesul de build și să își mențină o stare de funcționare consistentă în cazul apariției erorilor în timpul transmisiei de date în rețea.

Astfel, metodologia de întocmire a lucrării de cercetare propune realizarea unei serii de mai multe activități, pentru a explora modul de funcționare și utilitatea conceptului de compilare distribuită pentru proiecte dezvoltate folosind limbajele de programare C și C++:

- Studiul bibliografic - în vederea dezvoltării unui sistem de compilare distribuită, este necesară o analiză aprofundată a soluțiilor deja existente pentru realizarea acestui tip de sarcină. Studiul bibliografic oferă o imagine de ansamblu a principiilor ce stau la baza compilării distribuite, a implicațiilor practice ale aplicării unei astfel de abordări, precum și a cazurilor de utilizare cele mai frecvente pentru astfel de sisteme
- Proiectarea arhitecturii sistemului - aceasta reprezintă o etapă premergătoare implementării concrete și cuprinde descrierea comportamentului pentru modulele care intră în componența sistemului, cu modalitățile de funcționare și interacțiune aferente. De asemenea, se studiază posibilitatea aplicării unor concepte moderne pentru dezvoltarea sistemelor distribuite și se definește structura arhitecturii hardware și software a ansamblului
- Implementarea sistemului - pe baza arhitecturii definite în etapa anterioară, se implementează soluții practice pentru îndeplinirea operațiunilor asociate funcționării sistemului de compilare distribuită: prelucrarea datelor de intrare, balansarea sarcinii de compilare între nodurile participante, stocarea artefactelor rezultate, gestiunea comunicării dintre noduri pe baza unui protocol predefinit
- Crearea unui mediu de test - validarea funcționării corecte a sistemului dezvoltat poate avea loc doar în contextul utilizării unei configurații corespunzătoare, care să fie cât mai apropiată de condițiile de operare reale, în ceea ce privește dispozitivele hardware implicate, atât din punct de vedere al puterii de calcul, dar și al resurselor de rețea
- Analiza rezultatelor obținute - pe baza metricilor ce caracterizează performanțele și comportamentul sistemului se pot identifica principalele avantaje și dezavantaje ale abordării alese, raportate la celelalte alternative propuse în literatura de specialitate, dar și scenariile optime de utilizare pentru distribuirea procesului de compilare folosind mai multe sisteme de calcul

Pentru a restrânge aria de cercetare asupra conceptului de compilare distribuită pentru **C/C++**, nu se vor studia amănunțit toate aspectele adiacente funcționării unui astfel de sistem, precum testarea mai multor configurații de rețea, asigurarea funcționării compatibilității aplicației cu platform de rulare multiple sau optimizarea procesului de deployment.

Se urmărește dezvoltarea unui sistem distribuit care să fie compatibil cu sistemele de operare bazate pe **Linux** și uneltele software uzuale în dezvoltarea aplicațiilor C/C++, dintre care amintim: compilatoarele GCC și Clang, respectiv sistemele de build CMake, Ninja și GNU Make. Astfel, ne limităm doar la asigurarea interacțiunii facile a sistemului cu cele mai cunoscute medii de dezvoltare moderne.

De asemenea, funcționarea sistemului distribuit presupune apartenența sistemelor de calcul participante la o rețea de calculatoare locală LAN. Scenariul funcționării într-o rețea

de noduri distribuite asupra unei arii geografice însemnate implică necesitatea implementării unor măsuri de securitate suplimentare pentru păstrarea integrității și confidențialității datelor transferate, iar această activitate depășește sfera de cercetare a lucrării.

Totodată, o altă condiție care este impusă pentru funcționarea o constituie uniformitatea mediului de rulare pentru nodurile participante, având la bază presupunerea că sistemele de calcul implicate sunt dotate cu aceeași versiune de sistem de operare, sisteme de build și compilatoare, iar arhitecturile hardware ale acestora sunt identice. Nu se studiază metode de partiționare ale colecției de noduri de calcul pe baza compatibilității dintre platformele de rulare ale acestora.

După implementarea sistemului de compilare distribuită, un alt obiectiv important al lucrării îl constituie evaluarea procesului de utilizare a sistemului, având în vedere analiza următoarelor aspecte cheie:

- Durata de build - se măsoară timpul de compilare pentru mai multe proiecte open-source arhicunoscute, atât folosind un unic nod de compilare, pentru a se determina măsura în care utilizarea sistemului introduce overhead, raportat la durata de compilare locală pe nodul respectiv, precum și folosind multiple noduri care contribuie la procesul de compilare, pentru a măsura, în mod evident, îmbunătățirile aduse la eficientizarea procesului de build
- Comportamentul determinist - în mod ideal, comportamentul procesului de build, folosind sistemul dezvoltat, trebuie să fie același la fiecare rulare, iar erorile sau warning-urile generate de către compilator trebuie să fie cât mai apropiate de cele rezultate în urma procesului de compilare locală
- Randamentul paralelizării - în funcție de natura proiectului care este supus procesului de compilare, sistemul de compilare distribuită aduce grade diferite de îmbunătățire a timpilor de build. În scenariul ideal, structura proiectului permite o micșorare a timpului de compilare direct proporțională cu numărul de sarcini de compilare paralelă oferite de totalitatea nodurilor participante, însă în realitate, unele proiecte nu se bucură de îmbunătățiri semnificative folosind o astfel de abordare
- Volumul de date transferat - distribuirea procesului de compilare către mai multe sisteme de calcul poate să implice transferul unor volume de date semnificative, în rețea, ceea ce introduce necesitatea accesului la resurse de rețea adecvate, pentru a nu deteriora eficiența procesului de build din cauza curențelor de date de capacitatea de transfer dintr-o perioadă de timp sau latența de inițiere a transferului

Capitolul 3

Studiu bibliografic/Stadiul actual în domeniu

În cadrul acestei secțiuni, vom prezenta câteva dintre soluțiile deja existente pentru a distribui sarcina de compilare a unui proiect asupra mai multor sisteme de calcul ce comunică prin intermediul unei rețele.

3.1 Distcc

Distcc[9] oferă posibilitatea de a distribui sarcina de compilare între mai multe sisteme ce comunică în rețea, fără a fi nevoie ca sistemele participante să partajeze același sistem de fișiere, fără sincronizare de clock-uri sau să aibă acces la aceeași versiune a codului sursă.

Distcc funcționează prin distribuirea sarcinilor de lucru de la un sistem client, către nodurile server, care vor realiza compilarea. Inițial, se analizează comanda de compilare executată de către client, pentru a determina dacă sarcina poate fi distribuită, se alege nodurile care să deservească cererea, apoi se rulează local preprocesorul asupra fișierelor ce vor fi supuse compilării. Ulterior, nodurile server primesc request-uri ce conțin fișierele sursă preprocesate și returnează către client fișierele obiect rezultate în urma compilării sau o listă cu erorile de compilare, în cazul unui eșec.

Aplicația client este invocată ca un wrapper pentru compilator-ul obișnuit (GCC de obicei), de către script-urile de build rulate de **GNU Make**. Comunicare între client-ul Distcc și nodurile server se realizează prin protocoalele de rețea **TCP** sau **SSH**, folosind un set simplu de mesaje pentru a trimite cereri de compilare: mesajul pentru cerere conține versiunea de protocol utilizată, argumentele din linia de comandă pentru invocarea compilatorului și fișierele sursă, în timp ce răspunsul unui nod server returnează fișierele obiect aferente.

Distcc oferă posibilitate ca mesajele interschimbate între nodurile client și server să fie comprimate înainte de trimitere, pentru a reduce dimensiunea volumului de date tri-

mis, folosind metoda de comprimare **LZO**[10], care se concentrează asupra obținerii unei viteze cât mai bune pentru operația de decompresie. Micșorarea volumului de date folosind compresia, îmbunătățește throughput-ul de cereri transmise între participanții sistemului Distcc.

Planificarea pentru lansarea noilor sarcini de compilare folosește un mecanism simplu, prin care fiecare participant păstrează un fișier de stare, iar suprasolicitarea unui nod este împiedicată prin definirea unui număr limită de job-uri de compilare care pot fi soluționate simultan, care ia o valoare dependentă de numărul de procesoare fizice de care dispune fiecare nod. La inițializare, fiecare nod pornește numărul maxim de procese Distcc-server, care ulterior așteaptă inițierea unei conexiuni de către un client care va solicita compilarea. Pentru a putea coordona activitatea proceselor server care rulează simultan pe fiecare nod, se folosesc fișiere lock Unix, care sunt eliberate automat de către un proces în momentul terminării sale.

Un dezavantaj al sistemului distribuit Distcc îl constituie faptul că nu există niciun mecanism de estimare a timpului de compilare pentru un nod, deoarece această valoare nu este dependentă doar de dimensiunea codului sursă, ci și de aspecte precum complexitatea codului sursă sau specificațiile hardware ale nodului respectiv. Așadar, pătrunderea unui nod foarte lent în rețeaua de servere Distcc, poate încetini semnificativ procesul de build, deoarece nodul solicitant va trebui să aștepte ca fiecare nod server să își finalizeze sarcina, înainte de a primi fișierele obiect și de a invoca linker-ul.

Totodată, un inconvenient care nu trebuie neglijat este faptul că Distcc este o aplicație care rulează exclusiv pentru sisteme de operare de tip **Linux**, ceea ce nu rezolvă problema compilării distribuite pentru **Windows**.

La data prezentării inițiale a Distcc, autorul afirmă că micșorarea timpilor de build este o valoare ce se modifică liniar, odată cu numărul de core-uri de procesare implicate, cel puțin pentru un număr mic de core-uri (pentru 3 calculatoare, build-ul a fost de 2.8 ori mai rapid). În ceea ce privește scalabilitatea sistemului Distcc, trebuie luat în calcul că dintre toate sarcinile aferente procesului de build: execuție Makefile, preprocesare, compilare, linking, doar compilarea este distribuită la mai multe noduri [11]. Pentru a determina care este limita de paralelizare a procesului de build, trebuie consultată Legea lui Amdahl, vizibilă în Figura 3.1:

$$speedup_{max} = \frac{1}{f + \frac{1-f}{P}}.$$

Figura 3.1: Legea lui Amdahl

Chiar dacă numărul de procesoare **P** tinde spre infinit, aportul de performanță este limitat de porțiunea din proces ce nu poate fi paralelizată [11].

Figura 3.2 exemplifică modul de funcționare al Distcc.

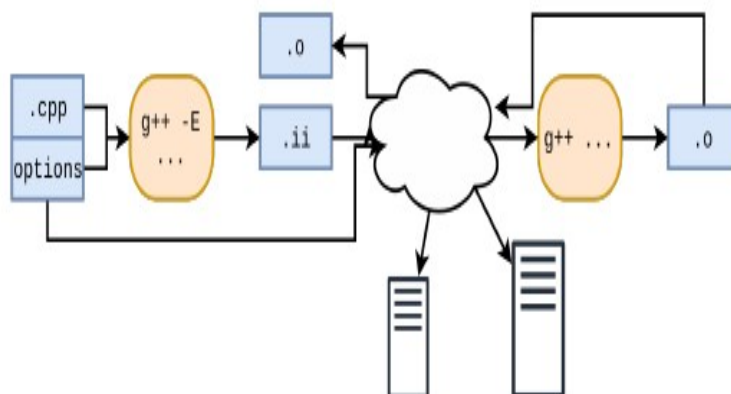


Figura 3.2: Funcționarea Distcc [12]

3.2 Ccache

Deși nu reprezintă un sistem distribuit, **Ccache**[13] este un mecanism caching pentru rezultatul compilării de fișiere sursă foarte des folosit alături de metoda descrisă anterior, Distcc, pentru a micșora și mai mult durata de build distribuit. Această combinație dintre **Distcc** și **Ccache** a fost studiată atent la Institutul CERN[12], întrucât procesul de build inițial pentru un proiect poate ajunge la 5 ore.

Sistemele tradiționale de build, precum **GNU Make** și **Ninja**, analizează timestamp-ul ultimei modificări a unui fișier sursă pentru a determina dacă compilatorul va fi invocat asupra unui fișier sursă sau nu. Problema apare în momentul schimbării de branch pentru sisteme de versionare precum **Git**, întrucât timestamp-ul fișierelor sursă își actualizează valoarea chiar dacă, conținutul nu se modifică, evenimentul fiind de natură să determine o recompilare completă a întregului proiect.

O proprietate interesantă a procesului de build pentru limbajele **C/C++** o constituie faptul că output-ul este complet determinat de conținutul surselor preprocesate și de argumentele de rulare ale compilatorului. Această proprietate este utilizată de către Ccache pentru a stoca output-ul compilării pentru un anumit fișier sursă, iar mai apoi să îl redea, în momentul începerii unui nou proces de build, evitând astfel apelarea inutilă a compilatorului pentru un fișier ce nu a fost modificat.

Sarcina principală pe care Ccache o îndeplinește o reprezintă determinarea existenței unui fișier object file asociat fișierului sursă de intrare, prin calcularea unei valori de dispersie bazată pe conținutul fișierului sursă și a flag-urilor folosite la compilarea inițială.

Ccache suportă mai multe moduri de operare, cel mai simplu dintre acestea fiind numit **preprocessor mode**, care presupune rularea preprocesorului asupra fișierului sursă

de intrare, iar mai apoi calcularea valorii hash pe baza conținutului unității de translație rezultate [14]. Dezavantajul acestui mod de funcționare este necesitatea de a apela întotdeauna preprocesorul asupra fișierului sursă de intrare, ceea ce încetinește lookup-ul.

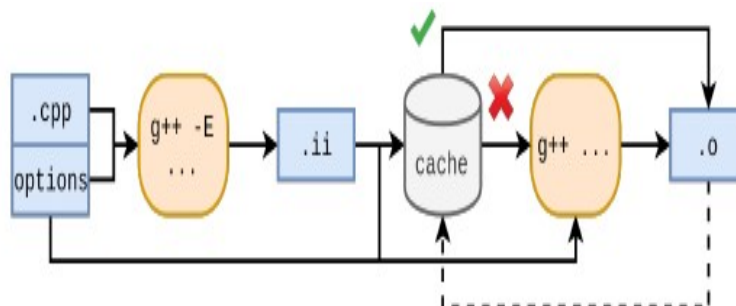


Figura 3.3: Ccache preprocessor mode [12]

De asemenea, Ccache poate funcționa într-un mod care evită pe cât posibil rularea preprocesorului, această acțiune fiind realizată doar în eventualitatea unui **cache miss**. În cadrul acestui mod de operare, valoarea hash este calculată direct pe baza fișierului sursă, fără preprocesare, iar valoarea este inclusă în cadrul unui fișier **manifest**, care conține referințe către rezultatele precedente al compilării (fișiere obiect, dependențe), care au fost produse pentru același hash, căi către fișierele header accesate la timpul compilării și valori checksum pentru fișierele incluse. În cazul unui cache miss, este necesară recalcularea valorii hash pentru fișierul de intrare și actualizarea manifest-ului.

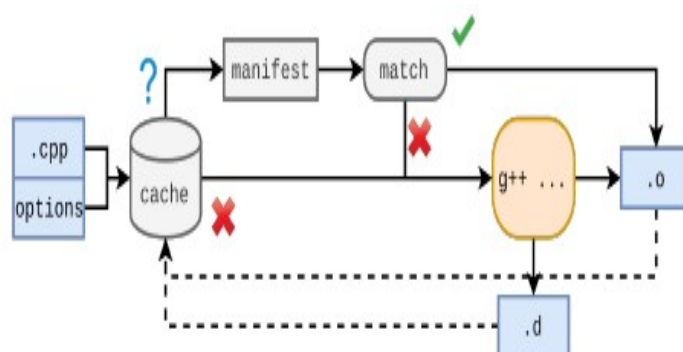


Figura 3.4: Ccache direct mode [12]

În cazul folosirii Ccache împreună cu sistemul distribuit Distcc, apare problema evidentă: cache-ul de compilare nu este distribuit pentru toate nodurile rețelei, ci fiecare nod participant are acces doar la cache-ul său local.

3.3 Sccache

Sscache[15] reprezintă varianta distribuită ca sistemului Ccache, care adaptează cache-ul inițial pentru a putea fi găzduit în cadrul unui serviciu cloud (sau pur și simplu remote) de stocare, precum: Amazon S3, Redis sau Cloudflare R2, ceea ce soluționează nevoia ca fiecare nod participant la procesul de compilare să dețină un cache local de fișiere obiect, care conține înregistrări doar pentru datele de intrare prelucrate de nodul respectiv.

Această soluție oferă un mecanism wrapper peste Ccache, astfel încât accesul la datele stocate remote să fie reglementat într-o manieră client-server [16]. Spre deosebire de Ccache, pentru calcularea unei valori hash se invocă întotdeauna preprocesorul, având astfel un comportament similar cu modul de operare **preprocessor mode**, dar se face offloading pentru volumul de date.

3.4 DiSCo

Dacă în cazul utilizării Distcc, singura dintre sarcinile din procesul de build care este paralelizată, este invocarea compilatorului asupra fișierelor preprocesate, **DiSCo** [17] propune o abordare inedită, în care preprocesarea, compilarea și link-editarea sunt efectuate în mod paralel, utilizând noduri remote.

Distcc utilizează schema de preprocesare locală, mult mai rudimentară, prin care nodul care solicită compilarea este singurul responsabil pentru realizarea etapei. Întrucât toată operațiunea are loc local, nu există problema ca un fișier de intrare să nu fie valabil, însă impune un cost însemnat asupra scalabilității sistemului. Pe de altă parte, preprocesarea remote, abordare aleasă de **DiSCo**, micșorează durata preprocesării, însă volumul de date tranzacționat între nodurile sistemului este mai mare, deoarece nu se mai trimite prin rețea doar fișierul preprocesat, ci atât fișierul sursă, cât și toate fișierele header asociate. O abordare asemănătoare adoptă și sistemul **IncrediBuild**[18], despre care nu sunt cunoscute multe detalii de funcționare, întrucât este o soluție comercială.

În ceea ce privește arhitectura sistemului de compilare distribuită **DiSCo**, la baza acestuia se află o serie de mai multe componente:

- Command Scheduler - modul care parsează conținutul unui **Makefile** pentru a determina potențialele dependențe dintre comenzile individuale de compilare, iar în funcție de relația dintre aceste comenzi, entitățile independente sunt grupate în blocuri de comenzi, elementele unui bloc fiind executate în paralel, iar mai multe blocuri de comenzi consecutive sunt executate secvențial; totodată, modulul command scheduler folosește o metodă de planificare statică pentru a determina spre ce nod al sistemului să fie trimis un anumit bloc de comenzi
- Connection Manager - folosește un mecanism de tip thread pool pentru a deschide sau termina conexiuni cu nodurile adiacente, folosite pentru comunicarea statusului pentru un proces de compilare în desfășurare [19]

- Compilation Manager - modul responsabil pentru executarea etapelor procesului de compilare
- Network Drive Manager - gestionează transferul de fișiere în mod implicit între nodurile sistemului și mediază interacțiunea cu **H-Cache-ul** nodului, responsabil pentru stocarea eficientă de fișiere header

Figura 3.5 ilustrează arhitectura sistemului distribuit de compilare DiSCo.

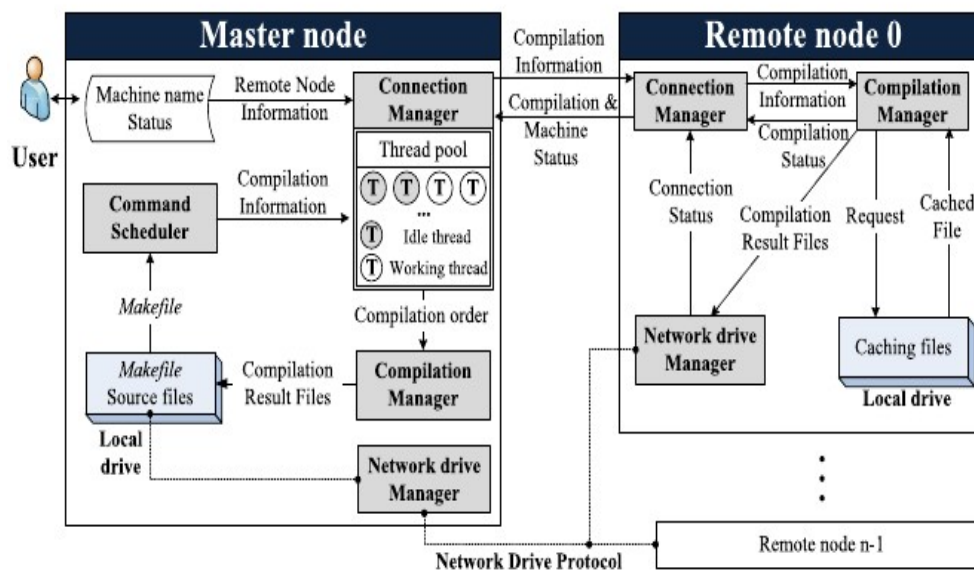


Figura 3.5: Arhitectura Disco [17]

Capitolul 4

Prezentarea proiectului

Acest capitol al lucrării are ca scop prezentarea principalelor concepte tehnice și arhitecturale care stau la baza implementării sistemului de compilare distribuită, descrierea deciziilor de proiectare care modelează structura sistemului și a soluțiilor practice propuse pentru etapa implementare. Sunt analizate în amănunt tehnologiile utilizate, pentru a crea o imagine de ansamblu a modului în care acestea influențează arhitectura sistemului.

4.1 Fundamentare teoretică

4.1.1 Sisteme distribuite

Abordarea calculului distribuit presupune utilizarea resurselor provenite de la mai multe sisteme de calcul interconectate pentru a contribui la rezolvarea sarcini, în mod colaborativ. Astfel, dificultatea sarcinii nu rămâne rezervată unui singur sistem de calcul, ci este partiționată și distribuită în mod eficient către mai multe sisteme, pe baza metodelor de balansare a sarcinii. Fiecare dintre nodurile participante își pune la dispoziție atât capacitatea de calcul, precum și resursele de memorie, iar comunicarea cu celelalte noduri participante, în scopul coordonării, se realizează de cele mai multe ori în cadrul unei rețele de calculatoare [20].

Astfel, paradigma de calcul distribuit oferă avantaje semnificative, în ceea ce privește randamentul de procesare a datelor și asigurarea desiponibilității serviciilor, către utilizatorii sistemului:

- Toleranță la eșecuri - prin mecanisme de monitorizare periodică a stării de disponibilitate a nodurilor din sistem, se pot remedia situațiile în care una dintre cererile ce trebuie soluționate nu poate fi deservită din cauza indisponibilității temporare a unui nod implicat, întrucât nodurile redundante ale sistemului pot prelua sarcina nefinalizată
- Folosirea eficientă a resurselor neutilizate - în general, un computer obișnuit poate să se afle în stare de inactivitate cam în 70% din timpul total de funcționare [20],

ceea ce înseamnă că aceste resurse computaționale nefolosite pot fi agregate eficient în cadrul unui sistem distribuit pentru a rezolva sarcini costisitoare, a căror realizare nu ar fi fezabilă folosind un singur sistem de calcul

- Balansarea volumului de lucru - în cazul în care un nod solitar din cadrul sistemului distribuit nu face față volumului de lucru, alte noduri de calcul pot fi alocate în mod dinamic pentru a prelua din sarcini, fără a introduce ineficiențe legate de suprasaturarea resurselor unui anumit nod
- Scalabilitatea - limitările introduse de utilizarea unui singur sistem de calcul, al cărui potențial de performanță maximă este finit, pot fi mitigate prin introducerea de mai multe noduri participante, în funcție de nevoi
- Distribuirea geografică convenabilă - întrucât nodurile participante ale sistemului comunică prin intermediul rețelelor de calculatoare, distribuirea nodurilor asupra unei arii geografice de mare întindere devine o posibilitate, iar pentru fiecare utilizator al sistemului se pot identifica resursele care sunt cel mai apropiate și implicit, cele mai facil de accesat, pentru a satisface solicitările

Pe de altă parte, implementarea paradigmei de calcul distribuit implică dezavantajul creșterii semnificative a gradului de complexitate pentru realizarea unui sistem fiabil, din cauza necesității de a asigura un mediu de comunicare stabil și performant, dar și a garanției că sistemul are o stare globală consistentă. Pentru simplul fapt că informațiile legate de starea sistemului sunt împărțite pe multiple noduri, există riscul să apară inconsistențe între informațiile reținute de fiecare nod [21].

De asemenea, idealul ca un sistem distribuit să se afle permanent într-o stare adecvată de funcționare se întemeiază pe asumții care nu sunt tot timpul adevărate:

- Lățimea de bandă a canalelor de comunicație dintre noduri este infinită
- Structura rețelei și a nodurilor este omogenă
- Rețeaua este sigură
- Rețeaua este tot timpul disponibilă
- Latența pentru transferurile de date dintre noduri este neglijabilă

În încheierea acestei secțiuni, oferim figura 4.1, pentru a ilustra structura generală a unui sistem distribuit, în care mai multe noduri comunică într-o rețea cu topologie **"bus"**.

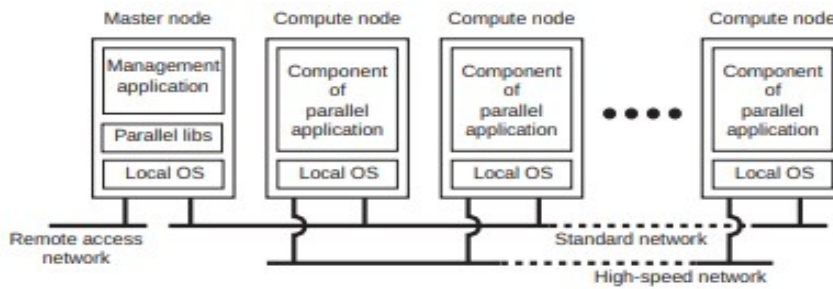


Figura 4.1: Sistem distribuit [21]

4.1.2 Mecanisme de concurență și paralelism

Una dintre problemele pe care orice sistem distribuit trebuie să le rezolve o reprezintă asigurarea capacității de comunicare simultană între un număr mare de noduri ale sistemului, pentru a oferi posibilitatea de scalare prin adăugarea de noduri suplimentare și a garanta că schimbul de informații dintre oricare două noduri nu trebuie să fie blocat până la finalizarea altor sarcini de calcul sau comunicare.

Având în vedere acest fapt, am ales să introduc în lucrare o secțiune ce prezintă fundamentele teoretice legate de asigurarea concurenței, în contextul dezvoltării sistemelor distribuite.

Concurența constituie capacitatea unui sistem de a executa mai multe sarcini în mod simultan. Până la momentul apariției sistemelor de calcul cu procesoare dotate cu mai multe nuclee, se crea impresia că mai multe sarcini de procesare erau soluționate în paralel prin schimbarea frecventă a contextului de execuție pentru sarcinile aflate în desfășurare [22]. În urma evoluției continue a resurselor hardware, au apărut mijloace multiple de asigurare a execuției concurente sau paralele.

Figura 4.2 are rolul de a ilustra diferența fundamentală dintre execuția concurentă și cea paralelă.

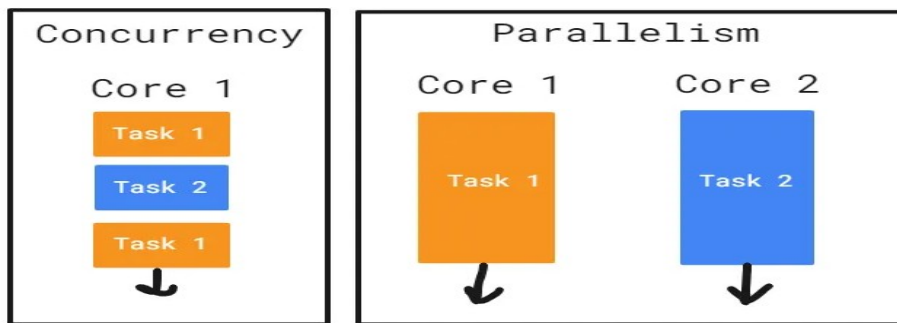


Figura 4.2: Concurență vs. paralelism [23]

Așadar, concurența presupune ca sistemul să asigure evoluția mai multor sarcini independente, care sunt active în același interval de timp și poate fi obținută prin comutare de context, fără a fi necesară utilizarea unui procesor cu mai multe nuclee sau a mai multor sisteme de calcul. Spre exemplu, concurența task-urilor este foarte utilă în momentul în care trebuie să așteptăm după un eveniment, fără a bloca execuția pentru restul programului. Mai sugestiv, toate aplicațiile server actuale trebuie să rezolve problema **C10K** [24], adică să asigure scalabilitatea aplicației pentru un număr mai mare de 10000 de conexiuni de rețea simultane, fără să epuizeze resursele computaționale. De cele mai multe ori, asigurarea concurenței este suficientă pentru a nu bloca firul principal de execuție al programului când se așteaptă finalizarea unei operațiuni **I/O** blocante ("I/O-bound"[22]).

Pe de altă parte, paralelismul oferă posibilitatea de a executa, în același moment discret de timp, două sau mai multe sarcini computaționale, fără a fi nevoie de executarea unei porțiuni din fiecare sarcină în mod secvențial pentru o perioadă redusă de timp. Principalul caz de utilizare al execuției paralele îl constituie împărțirea volumului de muncă între mai multe entități, cu scopul de a reduce durata de timp necesară pentru a finaliza procesarea ("CPU-bound"[22]).

În secțiunile următoare vom explica funcționarea câtorva dintre mecanismele de concurență și paralelism utilizate pentru dezvoltarea sistemului de compilare distribuită.

Thread-uri

Folosirea de multiple thread-uri[25] reprezintă o soluție eficientă pentru a rula mai multe task-uri în mod concurrent, în interiorul unui singur proces aflat în execuție. Una dintre diferențele principale dintre conceptul de thread și cel de proces, o constituie faptul că în vreme ce un proces își gestionează spațiul de adrese de memorie și resurse în mod total independent față de restul proceselor, mai multe thread-uri partajează aceleași resurse, ce aparțin procesului părinte.

Spre deosebire de procese, comunicarea între mai multe thread-uri active este mai facilă și mai eficientă, datorită spațiului de memorie partajat. De asemenea, împărțirea spațiului de memorie introduce necesitatea folosirii unor mecanisme de sincronizare între

firele de execuție (semafoare, excludere mutuală), pentru a împiedica coruperea datelor, datorată manipulării simultane a resurselor partajate, de către mai multe thread-uri care rulează în paralel.

Așadar printre cele mai utilizate mecanisme de sincronizare[22] pentru a asigura accesul sigur la resursele partajate, în contextul rulării în paralel a mai multor fire de execuție, le regăsim pe următoarele:

- **Mutex** - se utilizează pentru a asigura accesul exclusiv al unui singur thread la o porțiune critică din codul aplicației prin blocarea mutex-ului de către thread-ul care pătrunde în secțiunea critică. Celelalte thread-uri care urmează să execute codul din secțiunea protejată, trebuie să aștepte până când primul thread deblochează accesul la mutex
- **Semafor** - reprezintă o generalizare a conceptului de mutex, căci semaforul gestionează un contor numeric atomic, care descrie câte thread-uri se pot afla în regiunea de cod critică în mod simultan. Fiecare thread interacționează cu semaforul prin incrementarea sau decrementarea contorului acestuia, pentru a semnaliza celelalte thread-uri în legătură cu modificarea unei stări sau să aștepte ca alte thread-uri să părăsească regiunea critică. Folosind semafoare, se poate permite ca mai multe thread-uri să ruleze simultan cod din regiunea critică, până la un număr maxim de thread-uri
- **Variabila condițională** - mecanism de semnalizare între thread-uri care poate determina anumite fire de execuție să își suspende activitatea până la îndeplinirea unei condiții logice complexe. Thread-urile active pot să trimită semnale către variabila condițională, pentru a comunica thread-urilor care așteaptă, să își reia execuția

În cazul utilizării greșite a mecanismelor de sincronizare prezentate anterior, rularea mai multor fire de execuție în paralel determină apariția unor situații care deteriorează performanța aplicației și corectitudinea rezultatelor furnizate, posibil în mod nedeterminist:

- **Deadlock** - situație care apare când două sau mai multe thread-uri aflate în execuție se interblochează reciproc, întrucât fiecare dintre thread-urile în cauză reușește să obțină acces exclusiv la o resursă necesară pentru execuția unui alt thread, însă la rândul său, așteaptă ca accesul exclusiv la o resursă să fie eliberată. Această situație duce de obicei la blocarea pentru o perioadă nedeterminată de timp a thread-urilor aplicației
- **Livelock** - mai multe thread-uri rămân active pentru un timp nedeterminat, fără posibilitate de progres, întrucât încearcă fără succes să obțină acces la resurse
- **Înfometare (starvation)** - situație în care anumite thread-uri nu pot să își continue execuția, deoarece restul thread-urilor rulează în permanență și nu cedează accesul la resursele partajate după care așteaptă thread-urile înfometate

- Race condition - mai multe thread-uri accesează și încearcă să modifice starea unor resurse partajate în mod simultan, ceea ce generează stări impredictibile și eronate ale resurselor ce au fost modificate concurrent

Un alt aspect care merită menționat referitor la modul de funcționare al thread-urilor, care este luat în considerare și în proiectarea sistemului de compilare distribuită, îl constituie faptul că, operațiunea de creare și distrugere repetată a unui număr mare de thread-uri, de fiecare dată când o sarcină trebuie soluționată concurrent, duce la pierderi semnificative de performanță la rulare. Aceste pierderi de performanță se datorează apariției foarte frecvente a evenimentelor de comutare de context între threaduri și a faptului că un număr excesiv de thread-uri concurează să obțină acces asupra resurselor partajate.

Pentru a mitiga dezavantajele generate de crearea unui thread nou pentru fiecare cerere ce trebuie soluționată, se poate folosi metoda **"Eager Spawning"**[26], care presupune inițializarea unui număr predefinit de thread-uri la momentul pornirii aplicației. Aceste thread-uri sunt adăugate la o colecție, numită **"thread pool"**, responsabilă să preia execuția cererilor generate de utilizatori, fără a se aștepta după inițializare de thread-uri noi sau după comutări de context. Pool-ul de thread-uri poate să fie de dimensiune fixă sau redimensionat în mod dinamic în timpul rulării, pe baza numărului total de cereri în așteptare sau a capabilităților hardware ale sistemului de calcul care rulează aplicația.

Thread-urile care fac parte din colecție așteaptă inactive în perioadele în care nu există sarcini de soluționat, pentru a nu irosi resurse. În momentul în care este înregistrată o sarcină nouă, unul dintre thread-urile inactive este desemnat pentru a realiza execuția, iar în situația în care toate thread-urile din pool sunt deja ocupate, atunci sarcina este nevoită să aștepte până la eliberarea unui thread.

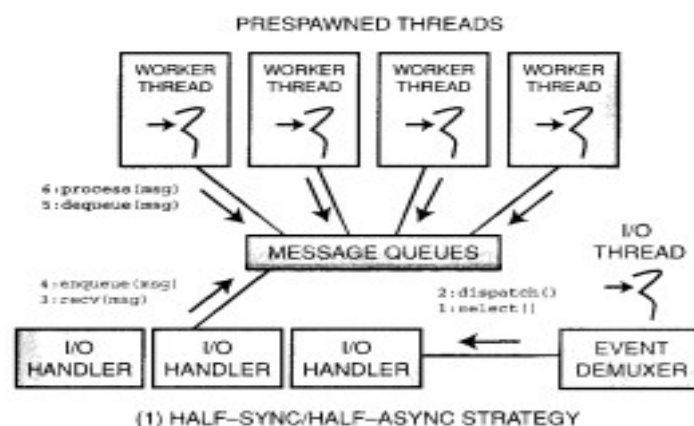


Figura 4.3: Concurență vs. paralelism [26]

Corutine

Rularea unei funcții obișnuite presupune execuția codului asociat de la începutul porțiunii de cod asociate, iar ieșirea din corpul funcției are loc o singură dată, moment în care apelul funcției este finalizat. O funcție nu stochează informații persistente de stare pentru rulare, motiv pentru care fiecare apel al funcției începe de la începutul blocului de cod asociat.

Corutinelor[27] reprezintă funcții care își pot sista temporar execuția când ajung la locații specifice din cod, iar execuția poate fi reluată ulterior de la locațiile în care a avut loc întreruperea. Pe durata întreruperii execuției, implementarea salvează informații de stare ale funcției înainte de suspendare, precum conținutul regiștrilor și valorile variabilelor locale de pe stivă. Capacitatea de a opri și a relua execuția înainte de finalizarea apelului oferă numeroase avantaje practice: spre exemplu, o corutină care trebuie să aștepte finalizarea unui apel blocant pentru a progresa poate să își suspende execuția pentru a permite altor funcții să ruleze, fără a bloca firul principal de execuție al aplicației. Astfel, corutinelor reprezintă un mecanism util pentru a asigura concurența aplicației, folosind un număr restrâns de thread-uri sau chiar un unic thread, care permite corutinelor să ruleze cod în mod cooperativ, prin suspendarea și reluarea execuției la momentul în care toate resursele devin accesibile.

În funcție de maniera de implementare a conceptului de "corutină", corutinelor se împart în corutine cu stivă proprie sau fără stivă ("stackless"). Pentru o corutină cu stivă proprie, toate variabilele locale pot să fie stocate persistent fără ca cel ce apelează corutina să aibă acces la valori. Pentru corutinelor fără stivă, toate valorile de pe stiva funcției sunt eliberate, ca pentru o finalizare obișnuită a unui apel de funcție, iar informațiile legate de starea de execuție a corutinei trebuie copiate într-o zonă de memorie auxiliară până la reluarea execuției. Corutinelor fără stivă prezintă avantajul că salvarea stării la momentul suspendării nu este obligatorie, ceea ce evită execuția suplimentară a a procesului de copiere a datelor de stare, însă nu mai este permisă suspendarea execuției corutinei în orice locație a blocului de cod asociat.

Spre exemplu, standardul **C++ 20** implementează corutinelor fără stivă proprie[28], iar starea de execuție, variabilele locale și parametrii sunt stocate într-un Coroutine Stack Frame, care se alocă în mod dinamic pe heap. Figura 4.4 ilustrează informațiile care sunt reținute la momentul suspendării execuției.

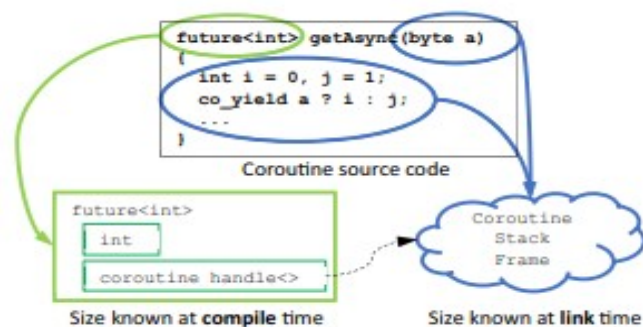


Figura 4.4: Coroutine stack frame [26]

4.1.3 Sisteme de build pentru C/C++

De cele mai multe ori, proiectele dezvoltate în C și C++ au în componență un număr foarte mare de fișiere sursă pe baza cărora se generează executabilul final, iar pe lângă numeroasele invocări ale compilatorului necesare în procesul de build al proiectului, se realizează o mulțime de operațiuni adiționale pentru a genera structuri de fișiere necesare pentru build, a localiza dependențele necesare sau a identifica care pași din procesul de build pot fi ignorați deoarece au fost efectuați deja cu succes. De cele mai multe ori, pentru a compila cu succes proiectul și a-l aduce într-o stare funcțională, este nevoie să se ruleze foarte multe comenzi diferite, ceea ce reprezintă o activitate aproape imposibilă dacă se dorește introducerea manuală.

Pentru a automatiza întreg procesul de build al proiectelor C/C++, se folosesc sisteme de build, al căror rol este cel de a transforma un limbaj de configurare expresiv și ușor de utilizat în secvența de comenzi ce vor fi rulate pentru produce artefacte în urma compilării proiectului.

Pe lângă faptul că utilizarea unui sistem de build scutește programatorul de la rularea manuală a comenzilor necesare, cele mai multe sisteme de build încearcă să reducă timpii de compilare ai proiectului determinând dacă fișierele sursă au suferit modificări de la ultima compilare, urmând să fie compilate doar fișierele ce au fost modificate. De asemenea, toate sistemele de build uzuale permit ca proiectul gestionat să fie compilat folosind opțiuni diverse de configurare, precum definirea unui anumit standard al limbajului C sau C++ care să fie folosit de compilator, prezența sau absența simbolurilor de debug, setarea nivelului de optimizare aplicat de compilator.

O particularitate comună a tuturor sistemelor de build pentru C/C++ o reprezintă faptul că permit definirea de mai multe configurații pe baza cărora să se realizeze compilarea, numite **"target"**[29]. În funcție de nevoile ce apar pe parcursul dezvoltării proiectului, proiectul va trebui compilat în configurații diverse, spre exemplu: build pentru toate configurațiile existente sau doar pentru o configurație anume, build ce injectează în sursele proiectului secvențe de cod folosite pentru măsurarea performanțelor la rulare,

build pentru suita de teste asociată proiectului, și așa mai departe.

În continuarea vom descrie succint modul de funcționare pentru câteva dintre cele mai cunoscute sisteme de build folosite pentru limbajele C și C++: **GNU Make**[29] și **CMake**.

GNU Make[29]

Utilitarul **make** primește ca date de intrare script-uri **Makefile**, în care sunt plasate instrucțiuni ce descriu pașii de urmat pentru realiza procesul de build pentru un proiect. GNU Make reprezintă un sistem de build universal și poate fi folosit pentru orice fel de limbaj, nu doar pentru C sau C++. Script-ul Makefile conține o serie de reguli care specifică modul în care fiecare fișier sursă trebuie compilat, având în vedere o listă de dependențe asociată fișierului sursă. Un fișier sursă este recompilat doar în momentul în care el însuși sau una dintre dependențele sale au suferit modificări.

Rularea utilitarului **make** trebuie efectuată în interiorul unui director care conține script-ul **Makefile**, menționat precedent, iar dacă nu se furnizează niciun target ca argument din linia de comandă, atunci GNU Make va realiza build-ul pentru primul target întâlnit în cadrul scriptului, care de obicei este responsabil pentru compilarea întregului proiect.

Figura 4.5 reprezintă un script Makefile generic, folosit pentru a exemplifica ulterior structura unui astfel de script.

```
edit : main.o kbd.o command.o display.o \  
      insert.o search.o files.o utils.o  
      cc -o edit main.o kbd.o command.o display.o \  
          insert.o search.o files.o utils.o  
  
main.o : main.c defs.h  
      cc -c main.c  
kbd.o : kbd.c defs.h command.h  
      cc -c kbd.c  
command.o : command.c defs.h command.h  
      cc -c command.c  
display.o : display.c defs.h buffer.h  
      cc -c display.c  
insert.o : insert.c defs.h buffer.h  
      cc -c insert.c  
search.o : search.c defs.h buffer.h  
      cc -c search.c  
files.o : files.c defs.h buffer.h command.h  
      cc -c files.c  
utils.o : utils.c defs.h  
      cc -c utils.c  
clean :  
      rm edit main.o kbd.o command.o display.o \  
          insert.o search.o files.o utils.o
```

Figura 4.5: Makefile[30]

La baza oricărui script Makefile se află **reguli (rules)**, care specifică ce target-uri trebuie compilate, care sunt condițiile necesare (prerequisites) pentru a declanșa compilarea

pentru target-uri, precum și comanda concretă care trebuie rulată pentru a compila target-urile sepcificate. O regulă are următorul format:

```
target: prerequisites
... comenzi de compilare ...
```

Cele mai uzuale elemente ale oricărui script Makefile sunt următoarele:

- Targets - reprezintă rezultatul de ieșire ce trebuie creat în urma compilării, fiind în general asociate unor fișiere obiect sau executabile. Un caz de target special îl reprezintă target-urile phony, care sunt asociate cu rularea unor comenzi specifice: build pentru tot proiectul sau ștergerea artefactelor rezultate din build-urile anterioare
- Prerequisites - fișierele sau țintele ce trebuie să existe sau să fi fost soluționate deja, pentru a putea rula ținta curentă. Dependințele ce au suferit modificări sunt reactualizate, iar ținta curentă este rulată din nou
- Lista de comenzi (Recipe) - comenzile din listă sunt rulate odată cu solicitarea unui build pentru ținta părinte, iar lista poate conține orice fel de comenzi care pot fi rulate în interiorul unui terminal
- Special targets - aceste target-uri au o semnificație specială și de cele mai multe ori sunt asociate cu anumite seturi de operațiuni, nu fișiere care trebuie compilate. Un target special este specificat folosind caracterul "." în fața numelui (spre exemplu)
- Prefixuri de comenzi - prin utilizarea de caractere speciale, se poate altera modul de rulare pentru comenzile asociate unei reguli: "-" - se ignoră erorile generate de rularea comenzii curente, "+" - se forțează rularea comenzii respective, "@" - nu se afișează în terminal fluxul de ieșire al comenzii

CMake[31]

Capitolul 5

Rezultate teoretice și experimentale

Împreună cu partea de prezentare a proiectului, trebuie să reprezinte aproximativ 70% din lucrare.

Aici sunt prezentate metodele teoretice sau practice de validare/verificare a soluțiilor propuse în partea anterioară, scenariile de testare a corectitudinii funcționale, a utilizabilității, performanței etc.

De asemenea, rezultatele testelor experimentale se pretează unor interpretări și comparații cu rezultatele altor metode similare.

Capitolul 6

Concluzii

6.1 Conținut

- un rezumat al contribuțiilor aduse
- a analiză critică a rezultatelor obținute: avantaje, dezavantaje, limitări
- o descriere a posibilelor dezvoltări și îmbunătățiri ulterioare

6.2 Detalii tehnice

6.2.1 Dimensiune

Cca 3–5% din total.

Bibliografie

- [1] J. R. Powell, “The quantum limit to Moore’s law”, *Proceedings of the IEEE*, vol. 96, nr. 8, pp. 1247–1248, 2008.
- [2] D. Kondo, B. Javadi, P. Malecot, F. Cappello și D. P. Anderson, “Cost-benefit analysis of cloud computing versus desktop grids”, în *2009 IEEE International Symposium on Parallel & Distributed Processing*, IEEE, 2009, pp. 1–12.
- [3] B. Stroustrup, *The C++ Programming Language*, 2013.
- [4] R. C. Martin, “Design principles and design patterns”, *Object Mentor*, vol. 1, nr. 34, p. 597, 2000.
- [5] D. Herity, “C++ in embedded systems: Myth and reality”, *Embedded Systems Programming*, vol. 11, nr. 2, pp. 48–71, 1998.
- [6] H. Rotithor, K. Harris și M. Davis, “Measurement and Analysis of C and C++ Performance”, *Digital Technical Journal*, vol. 10, nr. 1, pp. 32–47, 1999.
- [7] E. Burnette, *Hello, Android introducing Google’s mobile development platform: 2nd.* Android, 2009.
- [8] G. Lim, M. Lee, R. J. Lahaye și Y. I. Eom, “Distributed compilation system for high-speed software build processes”, în *2014 International Conference on Big Data and Smart Computing (BIGCOMP)*, IEEE, 2014, pp. 116–120.
- [9] M. Pool, *distcc, a fast free distributed compiler*, 2003.
- [10] W. Tarreau și D. Rodgman, *LZO stream format as understood by Linux’s LZO decompressor*, 2018.
- [11] S. Ashby, G. Eulisse, S. Schmid și L. Tuura, “Parallel compilation of cms software”, 2004.
- [12] R. Matev, “Fast distributed compilation and testing of large C++ projects”, în *EPJ Web of Conferences*, EDP Sciences, vol. 245, 2020, p. 05 001.
- [13] J. Rosdahl, *ccache - A fast compiler cache*, <https://ccache.dev/>, Accessed: 2025-02-09, 2025.
- [14] J. Rosdahl și Contributors, *ccache Manual: How ccache Works*, Accessed: 2025-02-09, 2025.

- [15] Mozilla și Contributors, *sccache - Shared Compilation Cache*, <https://github.com/mozilla/sccache>, Accessed: 2025-02-09, 2025.
- [16] T. Saario, “Enabling Sccache In CI System”, 2020.
- [17] K. Jo, S. W. Kim și J.-K. Kim, “DiSCo: Distributed scalable compilation tool for heavy compilation workload”, *IEICE TRANSACTIONS on Information and Systems*, vol. 96, nr. 3, pp. 589–600, 2013.
- [18] I. Ltd., *Incredibuild - Accelerate Development with Build Distribution*, <https://www.incredibuild.com/>, Accessed: 2025-02-09, 2025.
- [19] M. Mizuno, L. Chen și V. Wallentine, “Synchronization in a Thread-Pool Model and its Application in Parallel Computing.”, în *PDPTA*, Citeseer, 2003, pp. 1879–1885.
- [20] R. Buyya et al., “High performance cluster computing: Architectures and systems (volume 1)”, *Prentice Hall, Upper SaddleRiver, NJ, USA*, vol. 1, nr. 999, p. 29, 1999.
- [21] M. Van Steen, “Distributed systems principles and paradigms”, *Network*, vol. 2, nr. 28, p. 1, 2002.
- [22] J. Reguera-Salgado și J. A. Rufes, *Asynchronous Programming with C++: Build blazing-fast software with multithreading and asynchronous programming for ultimate efficiency*. Packt Publishing Ltd, 2025.
- [23] R. Kızılarşlan. “Parallel Programming with Modern Operating Systems I: Introduction Modern OS and Processors”. (2024), adresa: <https://medium.com/@recepzkizilarşlan/parallel-programming-with-modern-operating-systems-i-introduction-modern-os-and-processors-58582d77567e> (accesat în 25.6.2026).
- [24] H. Nenov și S. Todorov, “Asynchronous non-blocking IO model approach to avoid the problem C10K in web servers for static content”,
- [25] A. Williams, *C++ concurrency in action*. Simon și Schuster, 2019.
- [26] D. Schmidt și S. D. Huston, *C++ Network Programming, Volume I: Mastering Complexity with ACE and Patterns*. FT Press, 2001.
- [27] P. Diehl, S. R. Brandt și H. Kaiser, “Shared memory parallelism in modern C++ and HPX”, în *Workshop on Asynchronous Many-Task Systems and Applications*, Springer, 2023, pp. 27–38.
- [28] B. Belson, W. Xiang, J. Holdsworth și B. Philippa, “C++ 20 coroutines on microcontrollers—what we learned”, *IEEE Embedded Systems Letters*, vol. 13, nr. 1, pp. 9–12, 2020.
- [29] R. M. Stallman, R. McGrath și P. Smith, “GNU make”, *Free Software Foundation, Boston*, 1988.
- [30] R. M. Stallman, R. McGrath și P. D. Smith, *GNU make*. 2001.
- [31] R. Świdziński, *Modern CMake for C++: Discover a better approach to building, testing, and packaging your software*. Packt Publishing Ltd, 2022.

Anexa A

Pseudo-cod sau cod (dacă există)

```
/** Maps are easy to use in Scala. */
object Maps {
  val colors = Map("red" -> 0xFF0000,
                   "turquoise" -> 0x00FFFF,
                   "black" -> 0x000000,
                   "orange" -> 0xFF8040,
                   "brown" -> 0x804000)

  def main(args: Array[String]) {
    for (name <- args) println(
      colors.get(name) match {
        case Some(code) =>
          name + " has code: " + code
        case None =>
          "Unknown color: " + name
      }
    )
  }
}
```